

DEPARTMENT OF INFORMATION TECHNOLOGY



Secure Patient Data & Insurance Management System

Submitted By	Submitted To
Mohd Aadil	Prof. Bertilla Fernandes
Roll No.: 24BIT054 / 45 B.Sc. Information Technology (Third Year)	Department of Information Technology
Academic Year: 2026– 2027	Project Type: Web Application

CHAPTER 5 – IMPLEMENTATION

5.1 Implementation Approach: Project Summary

5.2 Coding Details and Code Efficiency

5.2.1 Code Efficiency

5.1 Implementation Approach:

The Secure Patient Data & Insurance Management System is implemented using Django and Python with PostgreSQL as the database. The system follows a modular approach where different functionalities are handled separately for patients, doctors, hospitals, billing, and insurance companies.

The implementation mainly includes database models, web views, forms, URL routing, authentication, validation, billing, and insurance claim processing.

5.1.1 Project Summary:

The project provides a centralized platform for managing patient information, medical records, hospital billing, and insurance claims. Different users are given access according to their roles.

The major implemented modules are:

- Patient Management
- Medical Record Management
- Billing Management
- Insurance Claim Management
- Authentication and Role-Based Access
- Document and Report Management

5.2 Coding Details and Code Efficiency:

The Secure Patient Data & Insurance Management System is developed using a modular coding approach. The project is divided into separate modules for patient management, medical records, billing, insurance claims, authentication, and user management. Django models are used for database operations, views handle the application logic, forms validate user input, and templates provide the user interface.

Reusable functions, Django components, and database relationships are used to reduce code duplication. Proper validation and access control are also implemented to improve the reliability and security of the system. Database queries are filtered to retrieve only the required records, which helps reduce unnecessary processing and improves application performance.

Database Models:

Django models define the structure of the database and establish relationships between different entities of the system. The project contains models such as:

- Patient
- MedicalRecord
- Billing
- InsuranceClaim
- InsuranceCompany

The models use relationships such as ForeignKey and OneToOneField to connect related healthcare information.

InsuranceClaim relationship:

Code:

```
class InsuranceClaim(models.Model):  
    patient = models.ForeignKey(  
        Patient,  
        on_delete=models.CASCADE,  
        related_name="insurance_claims"  
    )  
    medical_record = models.ForeignKey(  
        MedicalRecord,  
        on_delete=models.CASCADE,  
        related_name="insurance_claims"  
    )  
    bill = models.OneToOneField(  
        Billing,  
        on_delete=models.CASCADE,  
        related_name="insurance_claim"  
    )  
    insurance_company = models.ForeignKey(  
        InsuranceCompany,  
        on_delete=models.CASCADE,  
        related_name="insurance_claims"  
    )
```

Patient Management:

The Patient Management module allows authorized users to add, search, view, update, and manage patient records.

PatientViewSet:

Code:

```
class PatientViewSet(viewsets.ModelViewSet):
```

```
    queryset = Patient.objects.all()
```

```
    serializer_class = PatientSerializer
```

```
    permission_classes = [
```

```
        DjangoModelPermissions
```

```
]
```

```
    search_fields = [
```

```
        'patient_id',
```

```
        'first_name',
```

```
        'last_name',
```

```
        'phone_number',
```

```
        'email',
```

```
]
```

```
    ordering_fields = [
```

```
        'patient_id',
```

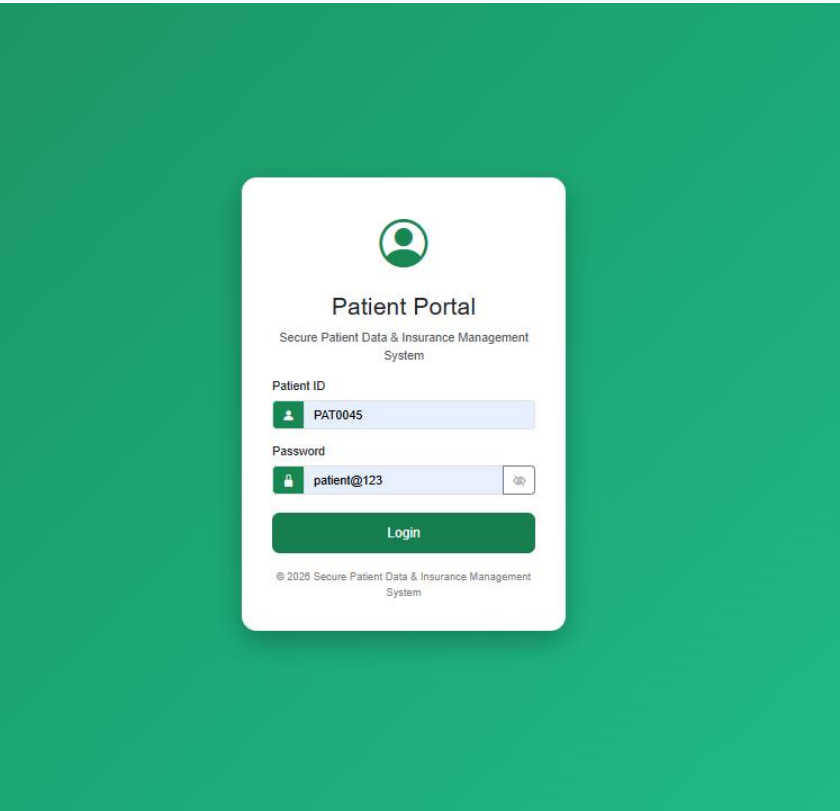
```
        'first_name',
```

```
        'last_name',
```

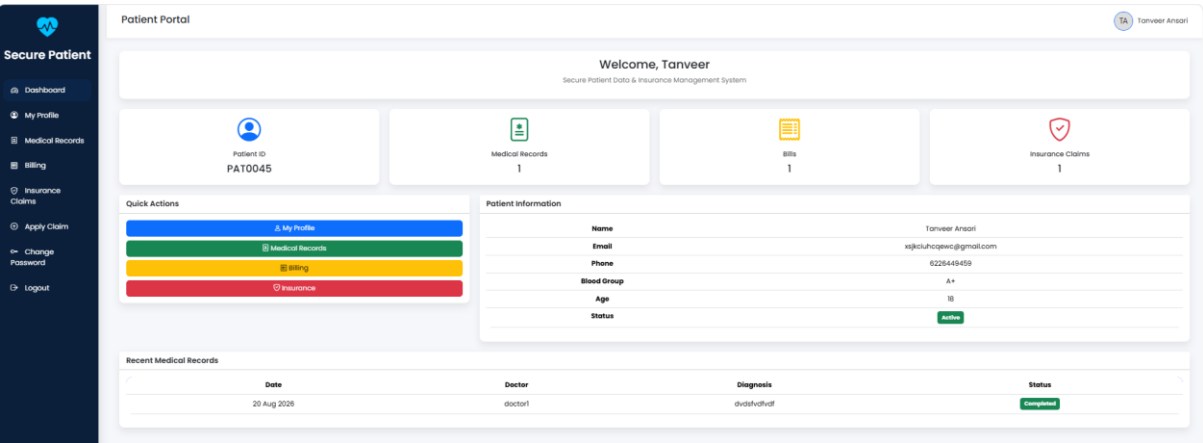
```
        'created_at',
```

```
        'date_of_birth',]
```

Patient login_page:



Dashboard:



My Profile:

Patient Portal

T Tanveer Ansari



Tanveer Ansari
Patient ID: PAT0045
Active

Personal Information

Patient ID PAT0045	Full Name Tanveer Ansari
Email xsjkcuhcqewc@gmail.com	Phone Number 6226449459
Gender Female	Date of Birth 01 Jun 2008
Blood Group A+	Address sjbcwe

Medical Record Management:

The Medical Record module stores diagnosis, treatment, prescription, department, doctor, visit date, and medical reports.

Code:

```
class MedicalRecordViewSet(viewsets.ModelViewSet):
```

```
    queryset = MedicalRecord.objects.filter(
        status=True
    )
```

```
    serializer_class = MedicalRecordSerializer
```

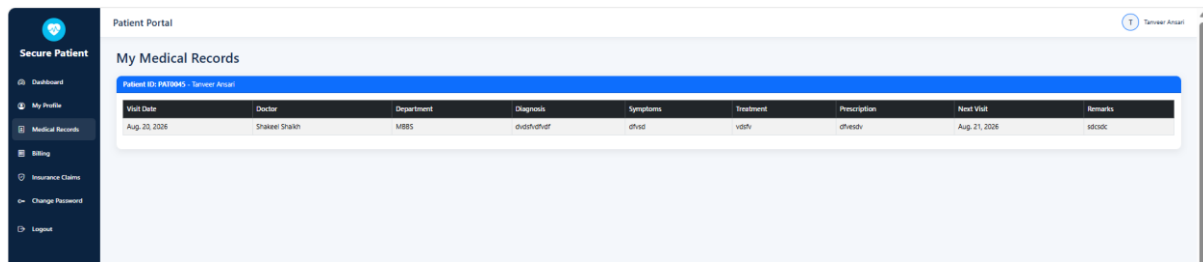
```
    filter_backends = [
        filters.SearchFilter,
        filters.OrderingFilter,]

```

```
    search_fields = [
        "patient__patient_id",
        "patient__first_name",
        "patient__last_name",
        "doctor_name",
        "department",
        "diagnosis", ]

```

Medical Records:



The screenshot shows a 'Patient Portal' interface. On the left is a dark blue sidebar with a 'Secure Patient' header and a list of menu items: Dashboard, My Profile, Medical Records (highlighted), Billing, Insurance Claims, Change Password, and Logout. The main content area is titled 'My Medical Records' and shows a table for 'Patient ID: 9876543 - Server Ansal'. The table has columns for Visit Date, Doctor, Department, Diagnosis, Symptoms, Treatment, Prescription, Next Visit, and Remarks. A single record is displayed for the visit on Aug 20, 2026, by Dr. Shaleel Shaikh in the MBBS department, with diagnosis 'cold/flu', symptoms 'cfsd', treatment 'vdrh', prescription 'chvedr', next visit on Aug 21, 2026, and remarks 'sdcac'.

Visit Date	Doctor	Department	Diagnosis	Symptoms	Treatment	Prescription	Next Visit	Remarks
Aug 20, 2026	Shaleel Shaikh	MBBS	cold/flu	cfsd	vdrh	chvedr	Aug 21, 2026	sdcac

Billing Management:

The Billing module manages consultation, medicine, laboratory, and other charges. The total bill amount is calculated automatically.

- Model calculates the total amount before saving the bill.

Code:

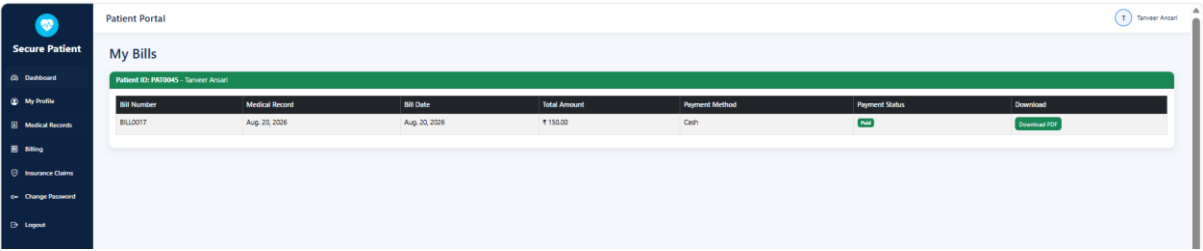
```
self.total_amount = (  
    self.consultation_fee  
    + self.medicine_fee  
    + self.laboratory_fee  
    + self.other_charges  
)
```

- Billing system also automatically generates a bill number:

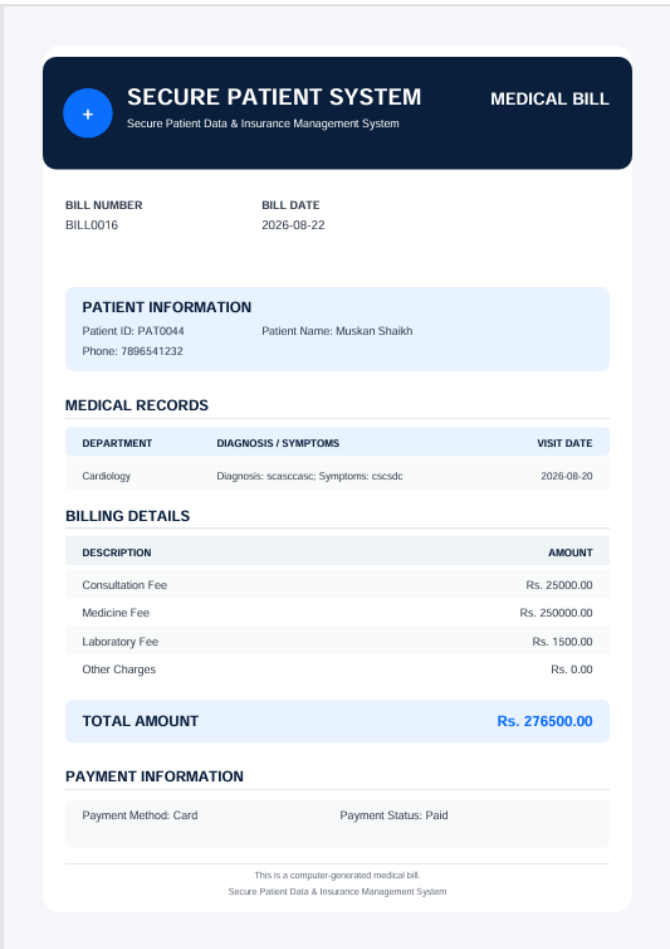
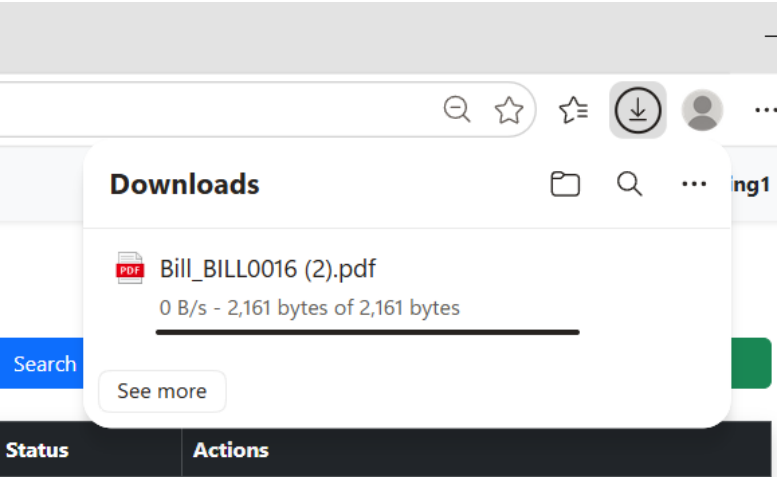
Code:

```
if is_new:  
    self.bill_number = f"BILL{self.pk:04d}"  
    Billing.objects.filter(  
        pk=self.pk  
    ).update(  
        bill_number=self.bill_number  
    )
```

Billing:



Download Bill:



Insurance Claim Management:

The Insurance Claim module allows patients to submit claims and insurance companies to review them.

The claim supports four stages:

- Pending
 - Under Review
 - Approved
 - Rejected
-
- InsuranceClaim model also validates the claim amount against the bill amount and checks whether the medical record and bill belong to the selected patient.

Code:

```
if self.bill_id and self.claim_amount:
```

```
    if self.claim_amount > self.bill.total_amount:
```

```
        raise ValidationError(
```

```
            "Claim amount cannot exceed total bill amount."
```

```
        )
```

- The claim ID generation:

Code:

```
if is_new:
```

```
    self.claim_id = f"CLM{self.pk:04d}"
```

```
    InsuranceClaim.objects.filter(
```

```
        pk=self.pk
```

```
    ).update(
```

```
        claim_id=self.claim_id
```

```
    )
```

Insurance Claim:



Insurance Portal

Secure Insurance Management

Dashboard

Insurance Claims

Approved Claims

Rejected Claims

Reports

Notifications

Profile

Change Password

Logout

Insurance Dashboard

Insurance Claims

Search and review patient insurance claims

Search

Status

Search Claim ID or Patient

All Status

Search

Reset

All Insurance Claims

Claim ID	Patient	Bill Number	Claim Amount	Approved Amount	Status	Applied Date	Action
CLM0011	PAT0045 Tanveer Ansari	BILL0017	₹ 100.00	₹ 50.00	Approved	20 Aug 2026	View PDF
CLM0007	PAT0041 Kishan Gupta	BILL0013	₹ 2000.00	₹ 0.00	Pending	19 Aug 2026	View PDF

Approved Claims:



Insurance Portal

Secure Insurance Management

Dashboard

Insurance Claims

Approved Claims

Rejected Claims

Reports

Notifications

Profile

Change Password

Logout

Insurance Dashboard

Insurance Claims

Search and review patient insurance claims

Search

Status

Search Claim ID or Patient

All Status

Search

Reset

All Insurance Claims

Claim ID	Patient	Bill Number	Claim Amount	Approved Amount	Status	Applied Date	Action
CLM0011	PAT0045 Tanveer Ansari	BILL0017	₹ 100.00	₹ 50.00	Approved	20 Aug 2026	View PDF
CLM0010	PAT0044 Muskan Shaikh	BILL0016	₹ 10000.00	₹ 5000.00	Approved	19 Aug 2026	View PDF
CLM0009	PAT0043 harshit vimal	BILL0015	₹ 3000.00	₹ 3000.00	Approved	19 Aug 2026	View PDF
CLM0006	PAT0040 Himanshu Gupta	BILL0012	₹ 2000.00	₹ 2000.00	Approved	17 Aug 2026	View PDF

Rejected Claims:



Insurance Portal

Secure Insurance Management

Dashboard

Insurance Claims

Approved Claims

Rejected Claims

Reports

Notifications

Profile

Change Password

Logout

Insurance Dashboard

Rejected Claims

View all rejected insurance claims

All Claims

Rejected Claims

Claim ID	Patient	Bill No.	Claim Amount	Reason / Remarks	Date	Action
No rejected claims found.						

Reports:



Insurance Portal

Secure Insurance Management

Dashboard

Insurance Claims

Approved Claims

Rejected Claims

Reports

Notifications

Profile

Change Password

Logout

Insurance Dashboard

Insurance Company

Insurance Company

Insurance Reports

Overview of insurance claim statistics



2

Pending Claims



1

Under Review



4

Approved



0

Rejected

Claim Summary

Total Claims	7
Pending Claims	2
Under Review	1
Approved Claims	4
Rejected Claims	0

Notifications:



Insurance Portal

Secure Insurance Management

Dashboard

Insurance Claims

Approved Claims

Rejected Claims

Reports

Notifications

Profile

Change Password

Logout

Insurance Dashboard

Insurance Company

Insurance Company

Notifications

Insurance Notifications



New Claim Submitted

Patient ID: PAT0041

Claim ID: CLM0007

Amount: ₹ 2000.00

Date: Aug. 19, 2026, 3:36 p.m.

Review Claim



New Claim Submitted

Patient ID: PAT0038

Claim ID: CLM0005

Amount: ₹ 29000.00

Date: Aug. 17, 2026, 5:54 p.m.

Review Claim

Profile:



Insurance Portal

Secure Insurance Management

Dashboard

Insurance Claims

Approved Claims

Rejected Claims

Reports

Notifications

Profile

Change Password

Logout

Insurance Dashboard

Insurance Company

ICICI Lombard

Company Profile

Insurance company account information

Insurance Company Information

Company	ICICI Lombard	Email	iciciinsurance@gmail.com
Registration	ICI-INS-002	Phone	9875463211
Contact Person	Amit Patel		

Authentication and Role-Based Access:

Django authentication is used to control access to the system. Different users such as patients, doctors, hospital staff, and insurance companies have separate portals and permissions.

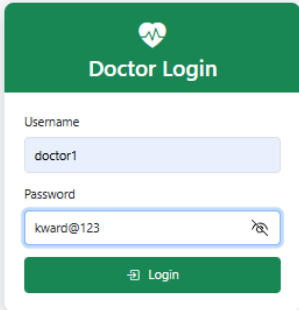
The system uses Django's `login_required` decorator and model permissions to restrict access to authorized users.

Code:

@login_required

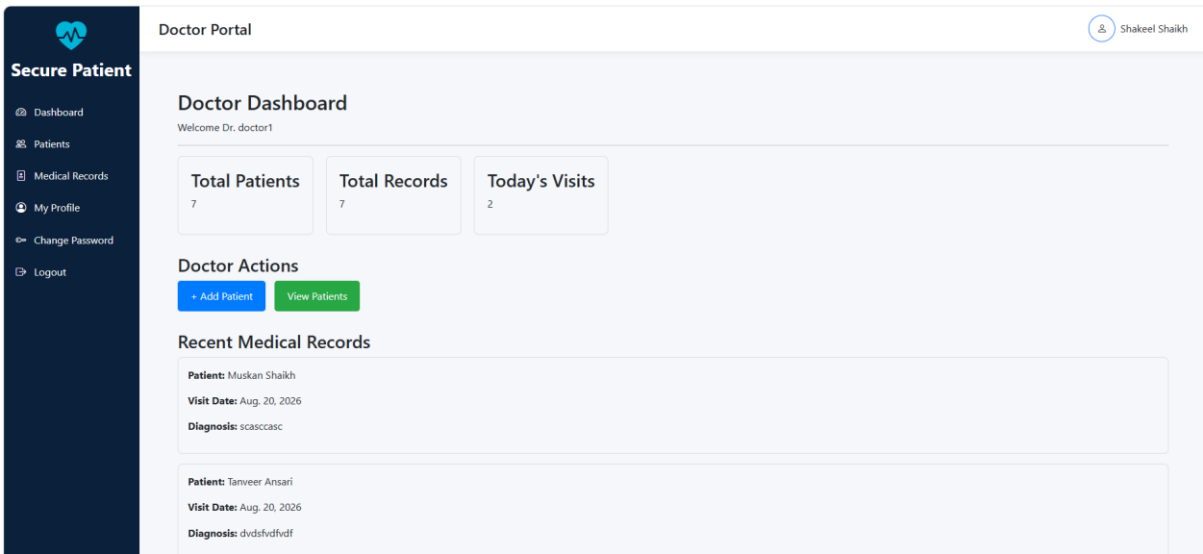
```
def doctor_profile(request):  
    return render(  
        request,  
        "doctor/profile.html",  
        {  
            "doctor": request.user,  
        },  
    )
```

Doctor login:



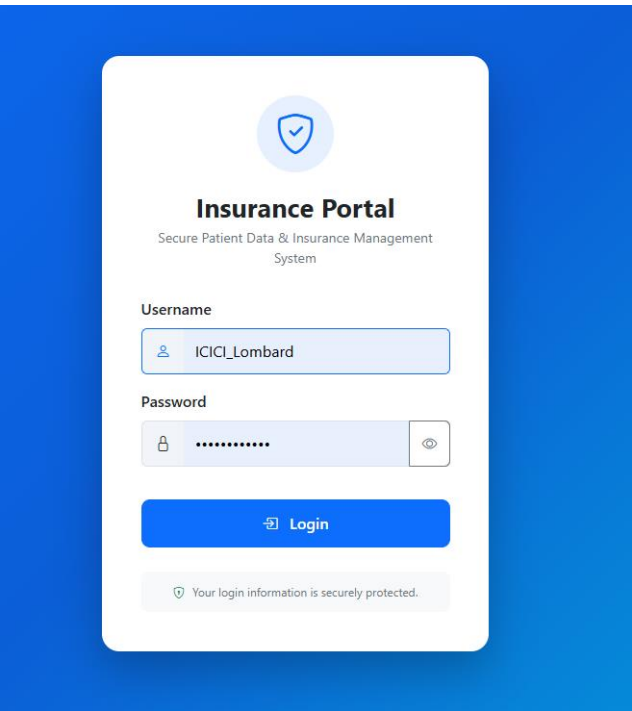
A screenshot of a 'Doctor Login' form. The form has a green header with a heart icon and the text 'Doctor Login'. Below the header, there are two input fields: 'Username' with the value 'doctor1' and 'Password' with the value 'kward@123'. The password field has a toggle icon on the right. At the bottom of the form is a green button with a white arrow icon and the text 'Login'.

Doctor Dashboard:

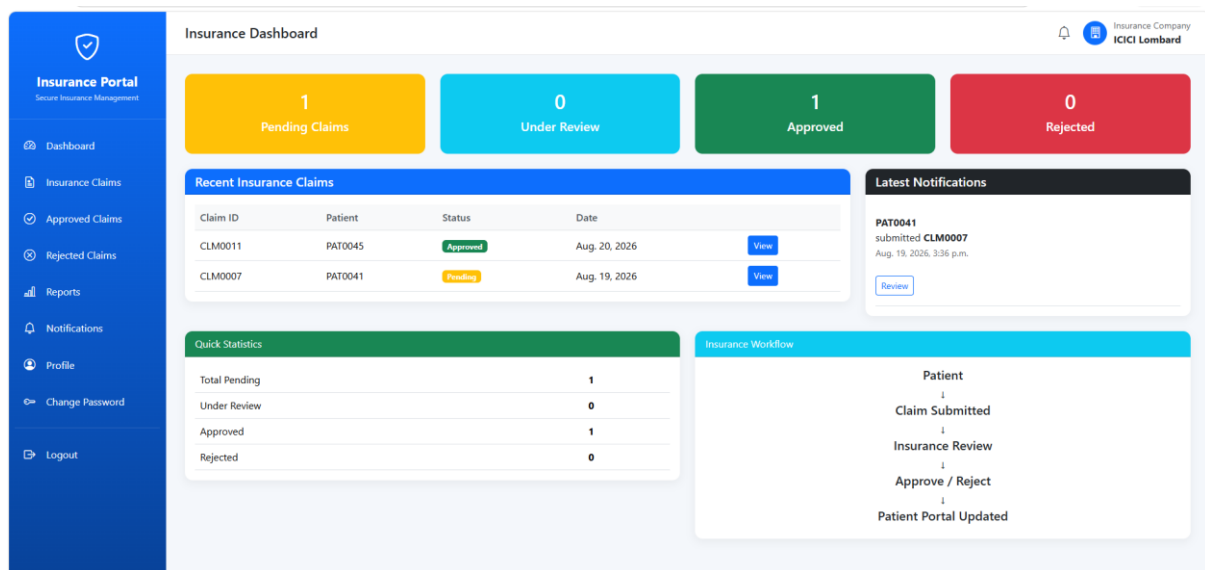


Insurance Portal:

ICICI Lombard Company:



Dashboard:



Form Validation:

Forms are used to collect and validate user input before storing information in the database.

the insurance claim form handles:

- Medical record
- Bill
- Insurance company
- Policy number
- Claim amount
- Reason
- Supporting document

The model also performs business-level validation using `full_clean()`.

Code:

```
def save(self, *args, **kwargs):
```

```
    self.full_clean()
```

```
    is_new = self.pk is None
```

```
    super().save(*args, **kwargs)Apply New Insurance Claim:
```

The screenshot shows a web application interface for a 'Patient Portal'. On the left is a dark blue sidebar with the title 'Secure Patient' and a list of navigation links: Dashboard, My Profile, Medical Records, Billing, Insurance Claims, Apply Claim, Change Password, and Logout. The main content area is light gray and titled 'Patient Portal'. In the top right corner of this area, there is a user profile icon with the name 'Tanveer Anwar'. Centered in the main area is a white modal form titled 'Apply New Insurance Claim'. The form contains several input fields: 'Select Insurance Company' (a dropdown menu), 'Select Medical Record' (a dropdown menu), 'Hospital Bill' (a dropdown menu), 'Enter policy number', 'Enter claim amount', 'Enter claim reason', 'Policy Number', 'Claim Amount', 'Reason for Claim', and 'Supporting Document' (a file upload area with a 'Choose File' button and a 'No file chosen' status). At the bottom of the form are two buttons: 'Apply New Claim' (in blue) and 'Cancel' (in gray). Below the form, there is a small text link: 'Upload medical reports, prescriptions, bills or other supporting documents.' At the very bottom of the page, there is a copyright notice: '© 2020 Secure Patient Data & Insurance Management System.'

5.2.1 Code Efficiency:

Code efficiency was considered to improve the execution speed, resource usage, maintainability, and overall performance of the system. The following techniques are used:

- **Code Reusability:** Common Django models, forms, templates, and functions are reused instead of writing the same code multiple times.
- **Efficient Database Queries:** Query filtering is used to retrieve only the required patient, medical, billing, and insurance records.
- **Reduced Database Processing:** Unnecessary records are avoided by using filters and appropriate query conditions.
- **Input Validation:** User input is validated before storing it in the database to prevent incorrect data and unnecessary errors.
- **Database Relationships:** ForeignKey and OneToOneField relationships are used to connect related data instead of storing duplicate information.
- **Query Filtering:** Active records and relevant claims are filtered before displaying or processing them.
- **Conditional Rendering:** Templates display information according to the user's role and available data.
- **Role-Based Access:** Users are provided access only to the modules and functions related to their role.
- **Modular Code:** Separate modules and components make the system easier to maintain, debug, and update.
- **Avoiding Duplicate Data Processing:** Data is processed only when required, reducing unnecessary computation.